

Desarrollo del Backend, Arquitectura Base y Modelo C4 del Sistema Multiplataforma

Autores Chiliguano Marco, Jaramillo Emilio, Sarango Ariana.
Fecha: 06 de nov. 2025

1. Introducción

El presente documento detalla la arquitectura técnica y la especificación de la API para el backend del Sistema de Miniworks El sistema ha sido diseñado utilizando un patrón de arquitectura de csr para garantizar la escalabilidad, mantenibilidad. ademas de agregar funciones de microservicios

2. Arquitectura General del Sistema

La arquitectura del backend de MiniWorks se basa en un patrón de microservicios. Cada dominio funcional —como gestión de usuarios, ofertas laborales, postulaciones, y empresas— se encapsula en un servicio independiente y autónomo, lo que permite escalabilidad, mantenimiento modular y despliegue individualizado.

La comunicación externa se gestiona mediante un API Gateway (Kong), que actúa como punto único de entrada (Single Point of Entry) para todas las solicitudes provenientes tanto de la aplicación web como de la aplicación móvil. Este componente proporciona enrutamiento, autenticación, seguridad y balanceo de carga, redirigiendo las peticiones al microservicio correspondiente según la lógica de negocio.

2.1. Tecnologías Principales

- **Lenguaje:** Node.js
- **Framework de API:** Express.js
- **Base de Datos:** MongoDB
- **API Gateway:** Kong
- **Contenerización:** Docker y Docker Compose
- **Autenticación:** JWT
- **Protocolo de Comunicación:** API RESTful utilizando JSON como formato de intercambio de datos.

3. Componentes del Sistema (Microservicios)

El sistema se descompone en los siguientes servicios, cada uno con una responsabilidad única y una base de datos aislada (esquema):

Servicio	Propósito	Base de Datos
api-gateway-kong	Punto de entrada único. Enruta peticiones a los servicios internos.	N/A

servicio-usuarios	Gestiona la información de estudiantes.	db_estudiantes
--------------------------	---	----------------

4. Especificación de Endpoints de la API

- **Módulo: Administrador**

-

Recurso	Método	Ruta	Descripción	Ejemplo de solicitud	Respuesta esperada
Listar todos los usuarios	GET	/api/admin/users	Obtiene la lista de usuarios (empleadores y estudiantes).	GET /api/admin/users	200 OK con JSON de usuarios.
Eliminar usuario	DELETE	/api/admin/users/{id}	Elimina un usuario del sistema.	DELETE /api/admin/users/12	204 No Content
Ver reportes o estadísticas	GET	/api/admin/report	Muestra métricas de trabajos publicados, postulaciones y usuarios activos.	GET /api/admin/report	200 OK con JSON de métricas.
Registrar Cualquier Usuario	POST	/api/admin/register	Registra un nuevo usuario	{ "email": "string", "password": "string", "name": "string", "role": "string" }	200 OK con JSON de usuarios.
Permite el ingreso del Admin	POST	/api/admin/auth/login	Permite el ingreso q	{ "email": "string", "password": "string" }	200 OK con JSON de ingreso Exitoso

- **Módulo: Estudiantes**

Recurso	Método	Ruta	Descripción	Ejemplo de solicitud	Respuesta esperada
Registro estudiante	POST	/api/students/register	Crea un nuevo estudiante.	POST → { "nombre": "Juan", "correo": "juan@unl.edu.ec", "password": "1234" }	201 Created
Ver trabajos disponibles	GET	/api/students/jobs/available	Retorna todos los trabajos activos.	GET /api/jobs/available	200 OK con lista de trabajos.

Postular a un trabajo	POST	/api/students/jobs/{id}/apply	Envía la postulación del estudiante.	POST /api/jobs/5/apply { "studentId": "string" }	201 Created
Ver historial de trabajos	GET	/api/students/{id}/history	Muestra los trabajos completados por el estudiante.	GET /api/students/10/history	200 OK con JSON del CV laboral.
Ver reseñas recibidas	GET	/api/students/{id}/reviews	Muestra las reseñas de empleadores.	GET /api/students/10/reviews	200 OK

Módulo: Empleadores

Recurso	Método	Ruta	Descripción	Ejemplo de solicitud	Respuesta esperada
Registro empleador	POST	/api/employers/register	Crea un nuevo empleador.	{ "nombre": "Taller López", "correo": "taller@gmail.com", "password": "abcd" }	201 Created
Publicar nuevo trabajo	POST	/api/employers/public/jobs	Publica un trabajo disponible.	{ "titulo": "Ayudante mecánico", "descripcion": "Limpieza y apoyo en taller", "pago": 25, "duracion": "4h" }	201 Created
Ver postulaciones recibidas	GET	/api/employers/{id}/applications	Lista las postulaciones recibidas en sus trabajos.	GET /api/employers/3/applications	200 OK
Aceptar postulante	PUT	/api/employers/applications/{id}/accept	Acepta al estudiante para el trabajo.	PUT /api/applications/7/accept	200 OK
Dejar reseña al estudiante	POST	/api/employers/students/{id}/review	Crea una reseña sobre el estudiante.	{ "calificacion": 5, "comentario": "Responsable y puntual" }	201 Created

Módulo: Trabajos

Recurso	Método	Ruta	Descripción	Ejemplo de solicitud	Respuesta esperada
---------	--------	------	-------------	----------------------	--------------------

Listar trabajos	GET	/api/jobs	Obtiene todos los trabajos publicados.	GET /api/jobs	200 OK
Buscar por palabra clave	GET	/api/jobs/search?query=limpieza	Busca trabajos relacionados.	GET /api/jobs/search?query=limpieza	200 OK
Ver detalle del trabajo	GET	/api/jobs/{id}	Muestra información de un trabajo específico.	GET /api/jobs/8	200 OK

5. Guía de Despliegue y Configuración

El despliegue del sistema se gestiona íntegramente a través de Docker y Docker Compose, asegurando un entorno de ejecución coherente y aislado.

5.1. Prerrequisitos

- Docker Engine
- Docker Compose

5.2. Configuración de Entorno

Cada microservicio requiere un archivo `.env` en su directorio raíz para cargar variables de entorno sensibles, como claves secretas y credenciales de base de datos.

Variables Críticas:

- **JWT_SECRET:** Clave secreta para la firma de tokens JWT. Debe ser idéntica en todos los servicios que validan tokens.
- **MONGODB_URL: PARA ACCEDER A LA DB**

5.3. Proceso de Inicio

El sistema se inicia ejecutando un único comando desde el directorio raíz del proyecto:

```
docker-compose up --build
```

Este comando realiza las siguientes acciones:

1. Construye (o reconstruye) la imagen de Docker para cada microservicio.
2. Inicia todos los contenedores definidos (API Gateway, servicios y base de datos) en una red interna aislada.
3. Expone los puertos públicos definidos (ej. `80` para el Gateway, `8001` para el Admin de Kong).

6. Ejecución de pruebas en Swagger

 **Swagger**
framework

MiniWorks API 1.0.0 OAS 3.0

Documentación de los endpoints RESTful del proyecto MiniWorks

[Contact Equipo MiniWorks](#)

Servers

Auth Endpoints de autenticación y usuarios

GET

/api/admin/auth/users

Obtiene todos los usuarios registrados

▼

GET

/api/admin/auth/users/{userId}

Obtiene el perfil de un usuario por ID

▼

POST

/api/admin/auth/register

Registra un nuevo usuario

▼

POST

/api/admin/auth/login

Inicia sesión de usuario

▼

GET

/api/admin/auth/report

Genera un reporte de usuarios

▼

Students Endpoints para gestión y acciones de estudiantes en MiniWorks

POST

/students/register

Registro de un nuevo estudiante

▼

GET

/students/{userId}/history

Ver historial de trabajos del estudiante

▼

GET

/students/{userId}/reviews

Ver reseñas del estudiante

▼

GET

/students/jobs/available

Ver trabajos disponibles para estudiantes

▼

POST

/students/jobs/{id}/apply

Postular a un trabajo

▼

Employers Endpoints para registro y gestión de empleadores

POST

/api/v1/employers/register

Registra un nuevo empleador

▼

POST

/api/v1/employers/public/jobs

Publica un nuevo trabajo

▼

GET

/api/v1/employers/{id}/applications

Obtiene las postulaciones recibidas por un empleador

📄 ▼

PUT

/api/v1/employers/applications/{id}/accept

Acepta un postulante

▼

POST

/api/v1/employers/students/{id}/review

Crea una reseña para un estudiante

▼

Jobs Endpoints para la gestión de trabajos en MiniWorks

GET

/jobs/all

Obtiene todos los trabajos publicados

▼

GET

/jobs/search

Busca trabajos por palabra clave

▼

GET

/jobs/{id}

Obtiene los detalles de un trabajo específico

▼

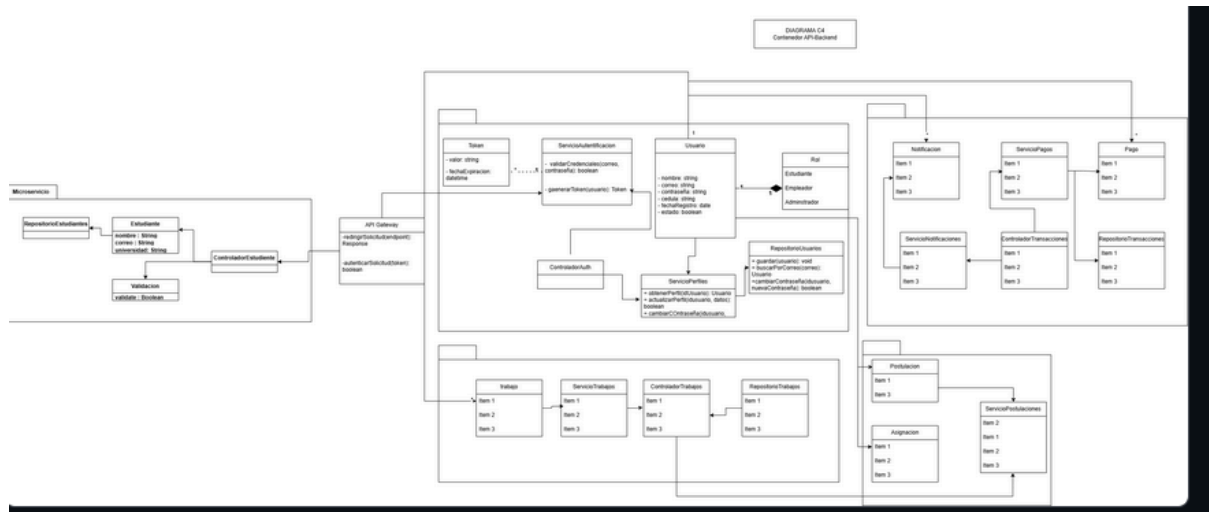
7. Configuración del entorno, dependencias y estructura modular del proyecto.

Se implementaron directorios para cada microservicio, el cual será independiente del resto.

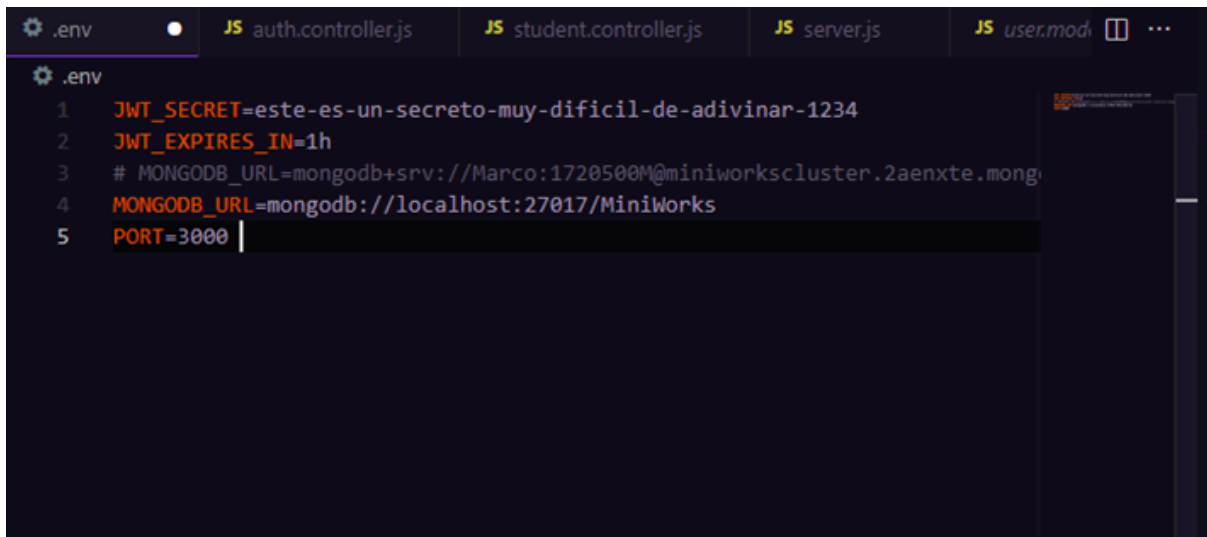
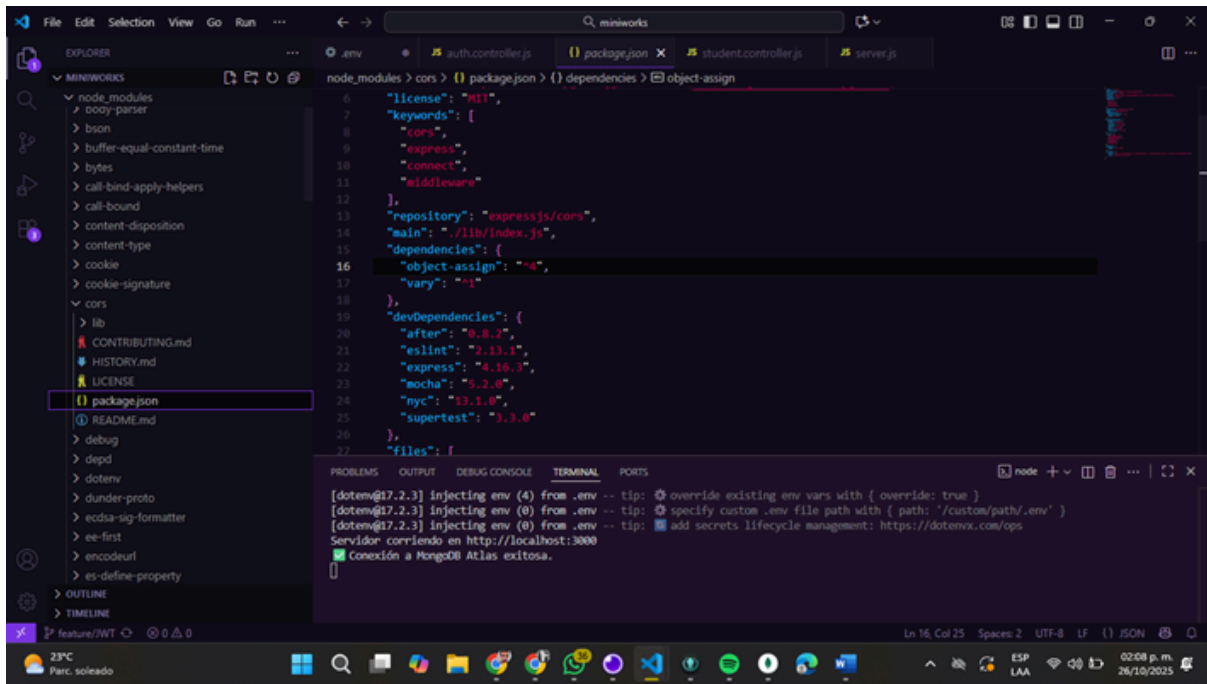
 MarcoChBuri trabajo y estudautes arreglados ce282c4 · 7 hours ago 		
 docs/architecture	Diagramas C4	5 days ago
 gateway	creacion de un mismo puerto para los d...	5 days ago
 src	trabjo y estudautes arreglados	7 hours ago
 .gitignore	Creacion de entorno	last month
 Dockerfile	creacion de un mismo puerto para los d...	5 days ago
 README.md	Update README.md	3 weeks ago
 package-lock.json	corecion de errores	5 days ago
 package.json	dependencias	2 days ago
 server.js	trabjo y estudautes arreglados	7 hours ago

Name	Last commit message	Last commit date
 ..		
 config	outentificacion	3 weeks ago
 docs	dependencias	2 days ago
 external	corecion de errores	5 days ago
 middlewares	jwt	5 days ago
 modules	trabjo y estudautes arreglados	7 hours ago
 shared	trabjo y estudautes arreglados	7 hours ago

8. Modelo C4



9. informe de seguridad



The screenshot shows the Visual Studio Code editor with the file explorer on the left. The file explorer shows a project structure with folders like 'src', 'config', 'external', 'middlewares', 'modules', 'students', 'shared', 'models', 'user.models.js', and 'repositories'. The 'modules' folder is expanded, showing 'auth', 'authcontroller.js', 'authservice.js', and 'perfileservice.js'. The 'authservice.js' file is selected and its content is displayed in the editor. The code defines an 'AuthService' class with a 'login' method. The 'login' method takes 'email' and 'password' as arguments, finds a user by email, checks the password, and returns a token if successful. The terminal at the bottom shows the command '[dotenv@17.2.3] injecting env (0) from .env' and the message 'Servidor corriendo en http://localhost:3000'.

```
require("dotenv").config();

class AuthService {
  async login(email, password) {
    const user = await userRepository.findBy(email);

    if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
      throw { status: 401, message: "Credenciales invalidas." };
    }

    const payload = {
      id: user.id,
      name: user.name,
      rol: user.rol,
    };

    const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '1h' });

    return {
      id: user.id,
      name: user.name,
      email: user.email,
      rol: user.rol,
      token,
    };
  }
}
```

[dotenv@17.2.3] injecting env (0) from .env -- tip: add secrets lifecycle management: https://dotenvx.com/ops
Servidor corriendo en http://localhost:3000
Conexión a MongoDB Atlas exitosa.

The screenshot shows the Visual Studio Code editor with the file explorer on the left. The file explorer shows a project structure with folders like 'src', 'config', 'external', 'middlewares', 'modules', 'students', 'shared', 'models', 'user.models.js', and 'repositories'. The 'modules' folder is expanded, showing 'auth', 'authcontroller.js', 'authservice.js', and 'perfileservice.js'. The 'authservice.js' file is selected and its content is displayed in the editor. The code defines an 'AuthService' class with a 'login' method and a 'verifyToken' method. The 'login' method takes 'email' and 'password' as arguments, finds a user by email, checks the password, and returns a token if successful. The 'verifyToken' method takes a 'token' as an argument and verifies it. The terminal at the bottom shows the command '[dotenv@17.2.3] injecting env (0) from .env' and the message 'Servidor corriendo en http://localhost:3000'.

```
class AuthService {
  async login(email, password) {
    const payload = {
      rol: user.rol,
    };

    const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '1h' });

    return {
      id: user.id,
      name: user.name,
      email: user.email,
      rol: user.rol,
      token,
    };
  }

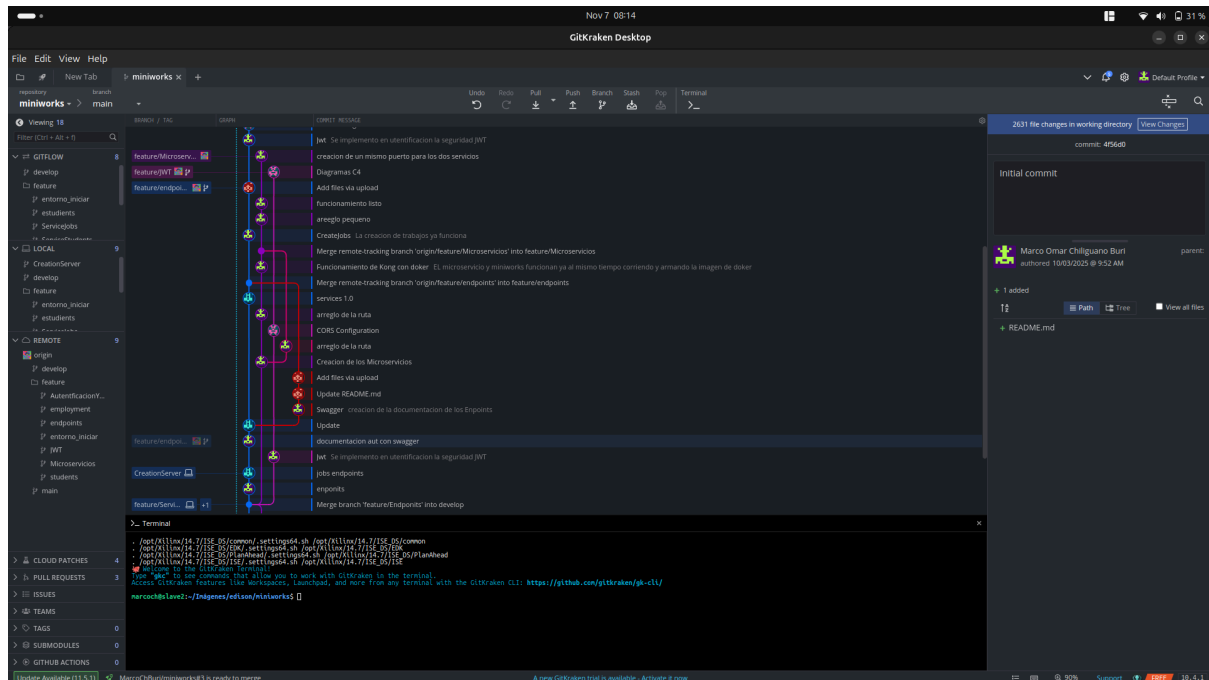
  verifyToken(token) {
    try {
      return jwt.verify(token, process.env.JWT_SECRET);
    } catch (error) {
      throw { status: 401, message: 'token inválido o expirado.' };
    }
  }
}
```

[dotenv@17.2.3] injecting env (0) from .env -- tip: add secrets lifecycle management: https://dotenvx.com/ops
Servidor corriendo en http://localhost:3000
Conexión a MongoDB Atlas exitosa.

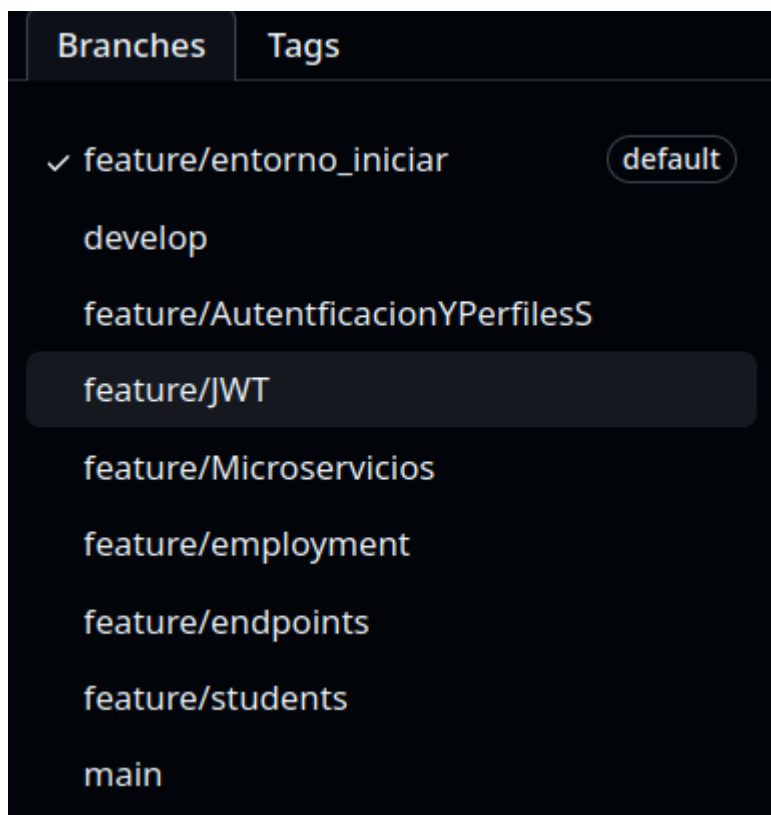
```
//CORS configuration
app.use(
  cors({
    origin: function (origin, callback) {
      if (!origin || allowedOrigins.includes(origin)) {
        return callback(null, true);
      }
      return callback(new Error('No permitido por CORS'));
    },
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization'],
    credentials: true,
    optionsSuccessStatus: 204,
  })
);
```

```
app.use((err, req, res, next) => {
  if (err.message.includes('CORS')) {
    console.error('Error de CORS:', err.message);
    return res.status(403).json({ error: 'Acceso no permitido por CORS' });
  }
  next(err);
});
```

10. Control de versiones con GitKraken y flujo GitFlow:



10.1 Ramas principales: **main**, **develop**, **feature/***.



10.2 Commits descriptivos y merges ordenados.

Commits on Oct 17, 2025		
outentificacion ...	 9b2e434	
 MarcoChBuri committed 3 weeks ago		
Merge branch 'feature/Endponits' into develop	 cf3fc0f	
 MarcoChBuri committed 3 weeks ago		
enponits	 8711344	
 MarcoChBuri committed 3 weeks ago		
jobs endpoints	 6385d96	
 ArianaSarango committed 3 weeks ago		
Commits on Oct 19, 2025		
documentacion aut con swagger	 3d88dd6	
 MarcoChBuri committed 3 weeks ago		
Update	 1f6bd9f	
 ArianaSarango committed 3 weeks ago		
Swagger ...	 7b16d1e	
 MarcoChBuri committed 3 weeks ago		
Update README.md	Verified  85341e3	
 MarcoChBuri authored 3 weeks ago		
Add files via upload	Verified  0fdd2d7	
 MarcoChBuri authored 3 weeks ago		